

4) How does duplicate login prevention improve security?

⇒ Duplicate login prevention blocks session hijacking.

If an unauthorized attacker somehow obtains a user's credentials, they cannot log in if the legitimate user is already active. ~~Add~~ Additionally, it maintains system integrity by preventing socket conflict, resource exhaustion and packet-routing synchronization errors within the server's client registry.

5) Suggest two additional security improvements for your application.

⇒ Two major security improvements for our application are:

a) Implementation of transport layer security.

(TLS/SSL): Wrapping our raw TCP sockets in an SSL layer would encrypt the entire conversation payload, defending against network sniffing (man-in-the-middle) via Wireshark.

b) Adding salt to password hashing: Appending a unique random string (called a "salt") to each user's password before running the SHA-256 algorithm.

This renders pre-computed dictionary attacks completely useless.

1) Explain the difference between authentication and authorization.

⇒ Authentication is the process of verifying 'WHO' a user is. In our application, this is done by checking the username and comparing the SHA-256 hash of the entered password against our secure database.

Authorization, on the other hand, determines 'WHAT' an authenticated user is allowed to do. For example, once authenticated, a user is authorized to send public broadcast message or private unicast message to active peers.

2) Why should passwords never be stored in plain text?

⇒ Storing passwords in a plain text is a severe security risk. If an attacker gains unauthorized access to the server, the file system or the database (user-credentials), they would instantly read all passwords. This compromises all user accounts immediately and enables "credential stuffing" attacks, where hackers try the same passwords on other public services.

3) What is the purpose of password hashing?

⇒ The purpose of password hashing is to perform a one-way cryptographic conversion (using SHA-256) on the password. A secure hash cannot be decrypted back into the plain text. The server only stores the hashed value. During login, the server hashes the input and compares the two hashes. If the database is compromised, the attacker only gets useless hashes keeping the real password safe.